

Class:	CPE 417	Semester:	Fall 2024
Project topic:	Security IOT System		
Student:	Kimberly Rojas		

1. Project description

a. General idea of your IoT system

My project is the security IoT system. This system will alert the user if there's an intruder in the home. The sensors that it will use are the ultrasonic sensor, flex sensor, and ambient light sensor. Some of the nodes that it will include are the human sensing node and the object sensing node. The communication protocol that will be used is MQTT to send and receive data. The cloud will process the data and make sure to send the user a notification for an intruder alert. The ThingSpeak will be able to display the data visually. The cloud will also be filtering the data and averaging the data. I will be using ThingSpeak.

b. System outcomes (profits to the end user)

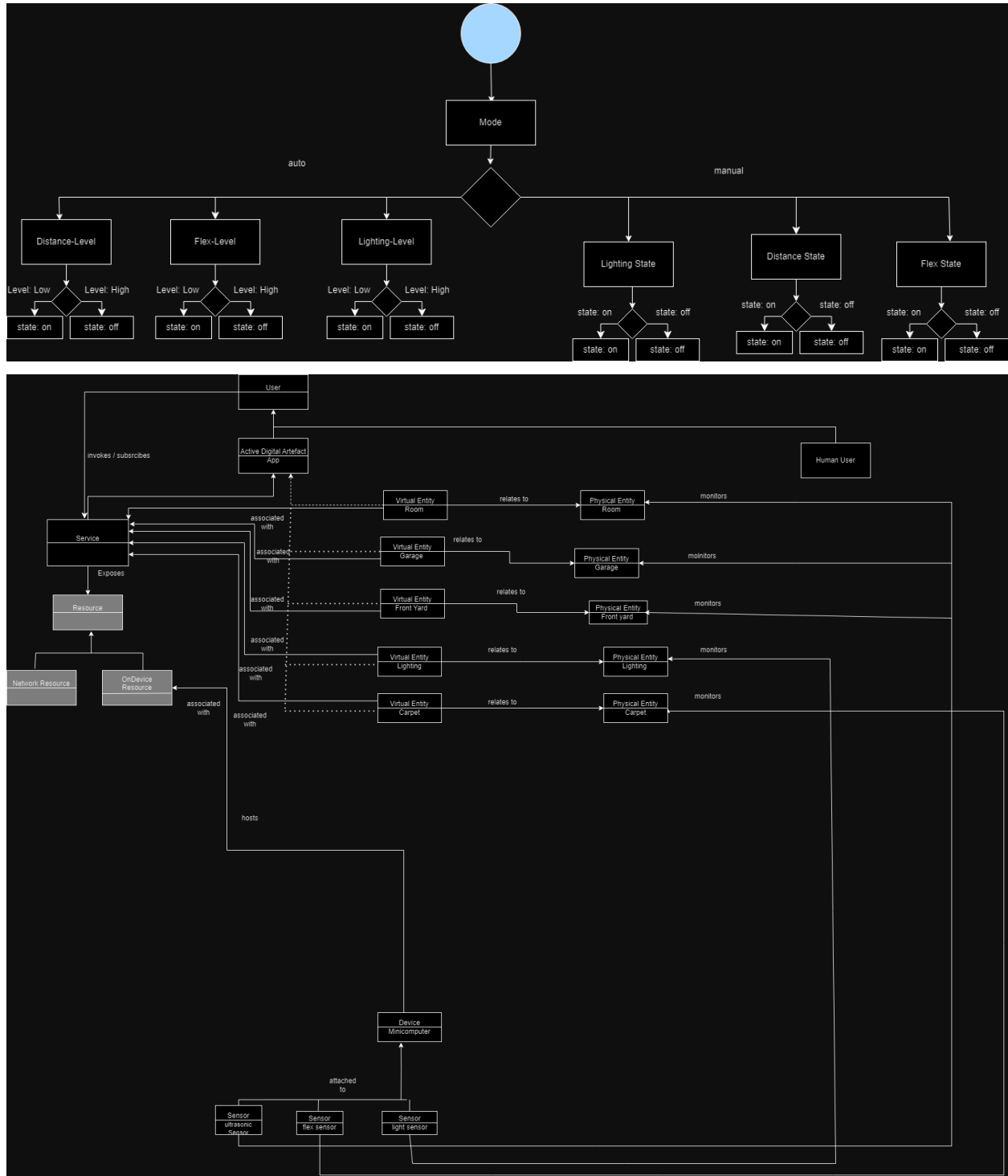
This system can alert the user in real time if there's any suspicious activity occurring. It also provides actual data visually from the sensors. This system also adds an extra level of security to the home. This will overall give the user a peace of mind and safeness during the day and night when their guard is off.

c. Technical description

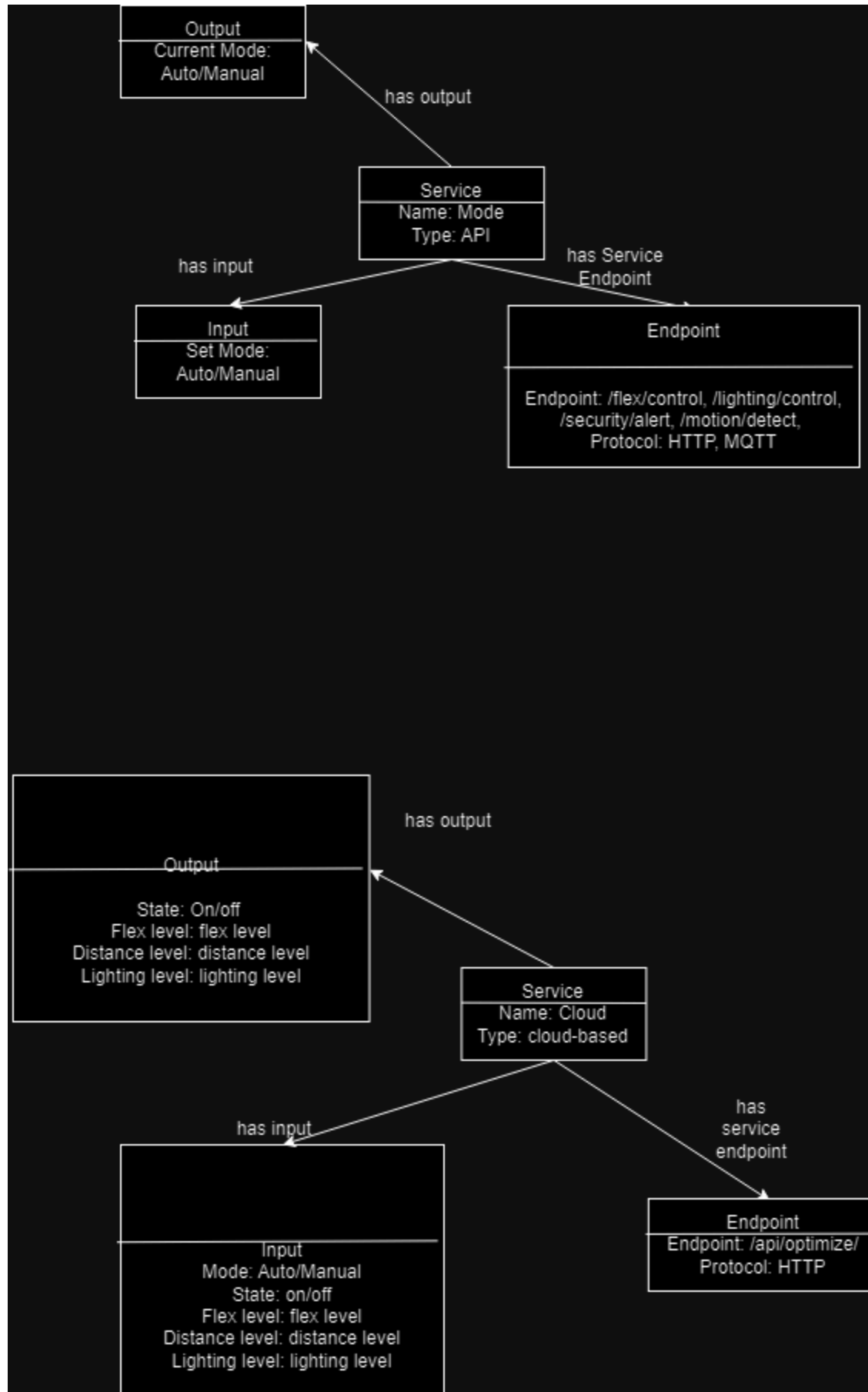
The ultrasonic sensor is used to detect any object in close proximity of the user or precious valuables. The sensor will send data of anything that happens to be really close by. This sensor can be used outside in the front yard or in the front door to detect when someone has broken in the home. The flex sensor is used to detect when an object or individual happens to step on the floor. The sensor will read a certain amount of bend that it detects and this could be used on the front of the door or in the entryway of the home to alert when someone has entered.

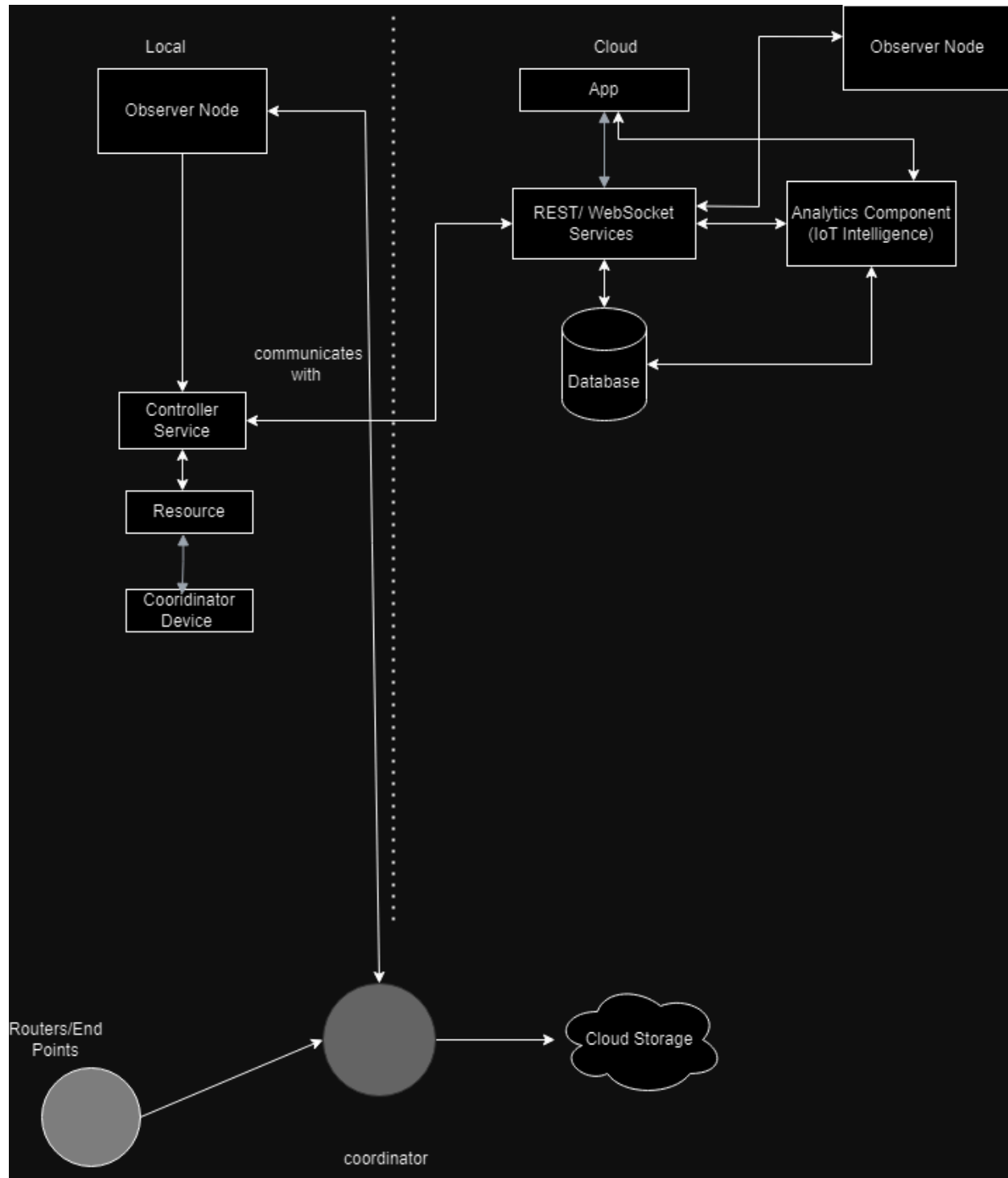
d. Diagrams

IoT Systems – Final project



IoT Systems – Final project





e. List of sensors

Vendor	Model	Comment
ElecFreaks	HC-SR04	Ultrasonic Sensor

Vishay Semiconductor	TEMT6000	Light Ambient Sensor
SparkFun	SEN-10264	Flex Sensor

f. List of nodes

Vendor	Model	Comment
Arduino	ABX00004	Arduino MKR 1000

g. List of other hardware components

Vendor	Model	Comment
SparkFun	large breadboard piece	breadboard
SparkFun	jumper wires 6"	wires
Digi-Key	2k	resistors

h. Selected cloud service

The cloud service I am using is ThingSpeak. The ThingSpeak is a program that I am familiar with. This program allows the publisher code to send in the sensor data to the cloud. It allows the cloud to display through a chart of all the sensor data. This display is mobile and can be viewed anywhere at any time. The ThingSpeak also allows for data to be processed and filtered so as to prevent any redundancies or false positives. It also allows the cloud to send alerts to the user which is crucial for this iot system. I also used the Blynk cloud service in order to display to the user the sensor reading data mobile.

2. Project operation

a. Describe how your project works

I have two Arduinos connected, one is to my laptop and the other is to my computer. The computer runs the main publisher code. The arduino connected to the computer is also connected to the flex sensor, ultrasonic sensor, and the ambient light sensor. The light sensor is connected to the breadboard and also has three connections: ground, vcc, and input pin. The flex sensor is connected to the breadboard and has 2 connections: two 2k resistors and vcc. The two 2k resistors are connected to the input of the flex sensor and to the ground that connection from the flex sensor is also to the input pin where it sends data. The ultrasonic sensor has 4 connections:

ground, echo pin, trigger pin, and vcc. I have the echo and trigger pin in its specific pin on the arduino board.

b. Describe what's the data flow in your project

The sensors all collect data from its surroundings. Once collected it verifies that a certain threshold is not met; if it is it will print the appropriate response to the serial monitor in order to alert the user. It will also send this data to the cloud and to the MQTT. The publisher code will send this data to the client code. The client is able to view all the sensor data being sent from the publisher. The publisher contains all the sensor data and basically sends it to the cloud and to the client. The broker is hosted on the test.mosquitto.org broker. I used three MQTT channels which are sensors/distance, sensor/flex, and sensors/light. The cloud's responsibility is to receive data from the MQTT channels. It filters out useless data in order for the sensor readings to be accurate. It is able to send the user real time data and display it for the user. It makes sensor readings more accessible to the user. The publisher also sends data to the Blynk software to display.

c. Describe cloud operation, including what's the data processing mechanism

The ThingSpeak filters the data when the publisher sends the sensor data. It uses a mechanism of obtaining data every 15 seconds. The ThingSpeak stores the sensor data over time. The ThingSpeak and Blynk also both visually display the data. The MQTT helps to transmit the data to ThingSpeak.

3. Screenshots / pictures

a. Publisher(s) operation (data being sent to MQTT broker)

IoT Systems – Final project

```
21:14:18.808 -> Light: 21
21:14:18.808 -> Sending message to topicLight: sensors/light
21:14:18.808 -> 21
21:14:26.795 -> Sending message to topicDistance: sensors/distance
21:14:26.795 -> 112.17
21:14:26.795 -> Sending message to topicFlex: sensors/flex
21:14:26.795 -> 22
21:14:26.795 -> Suspicious activity
21:14:26.795 -> Light: 23
21:14:26.795 -> Sending message to topicLight: sensors/light
21:14:26.795 -> 23
21:14:34.778 -> Sending message to topicDistance: sensors/distance
21:14:34.778 -> 113.07
21:14:34.778 -> object detected on floor
21:14:34.778 -> Sending message to topicFlex: sensors/flex
21:14:34.778 -> 9
21:14:34.778 -> Suspicious activity
21:14:34.778 -> Light: 28
21:14:34.778 -> Sending message to topicLight: sensors/light
21:14:34.778 -> 28
```

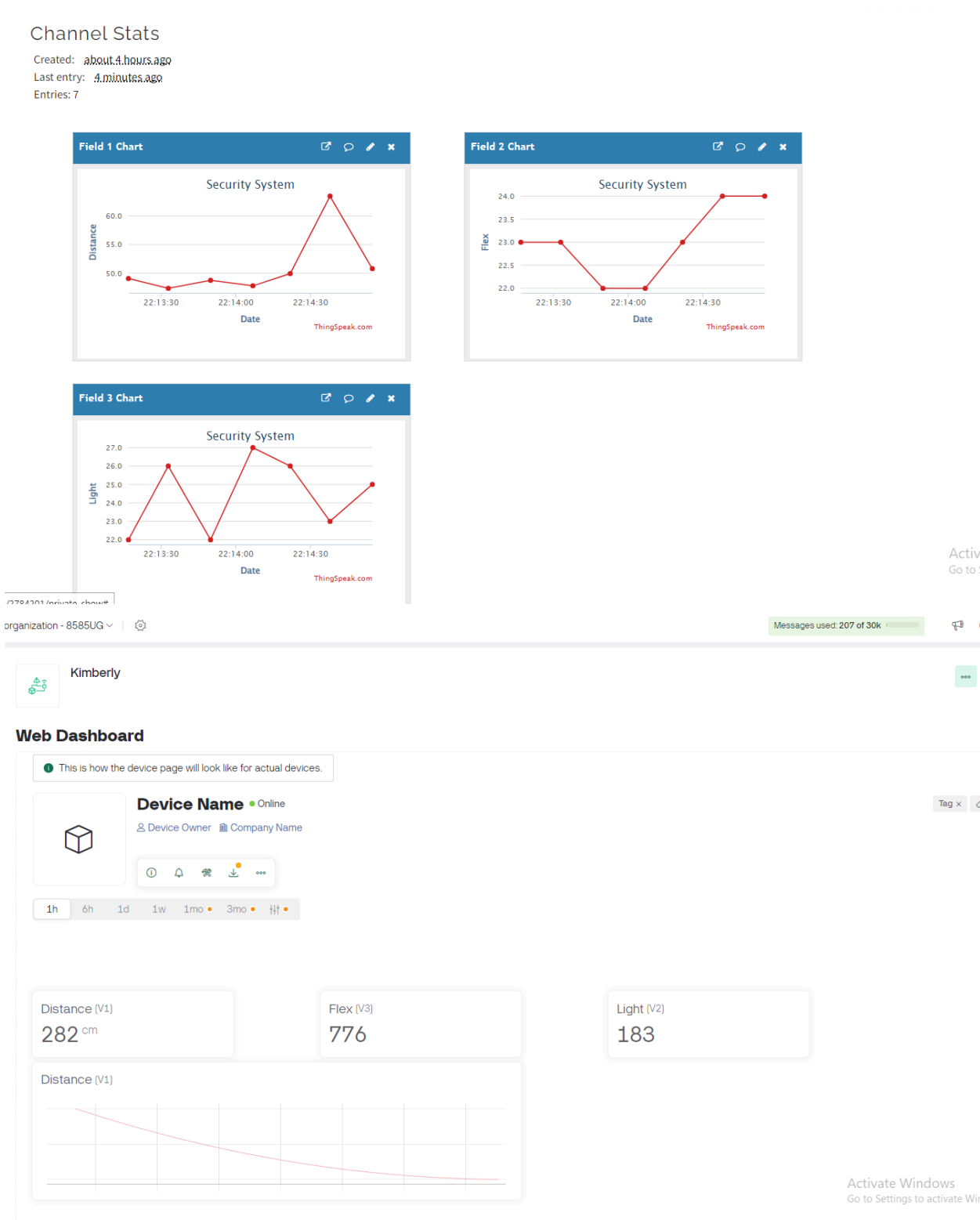
b. Subscriber(s) operation (data being received from MQTT broker)

IoT Systems – Final project

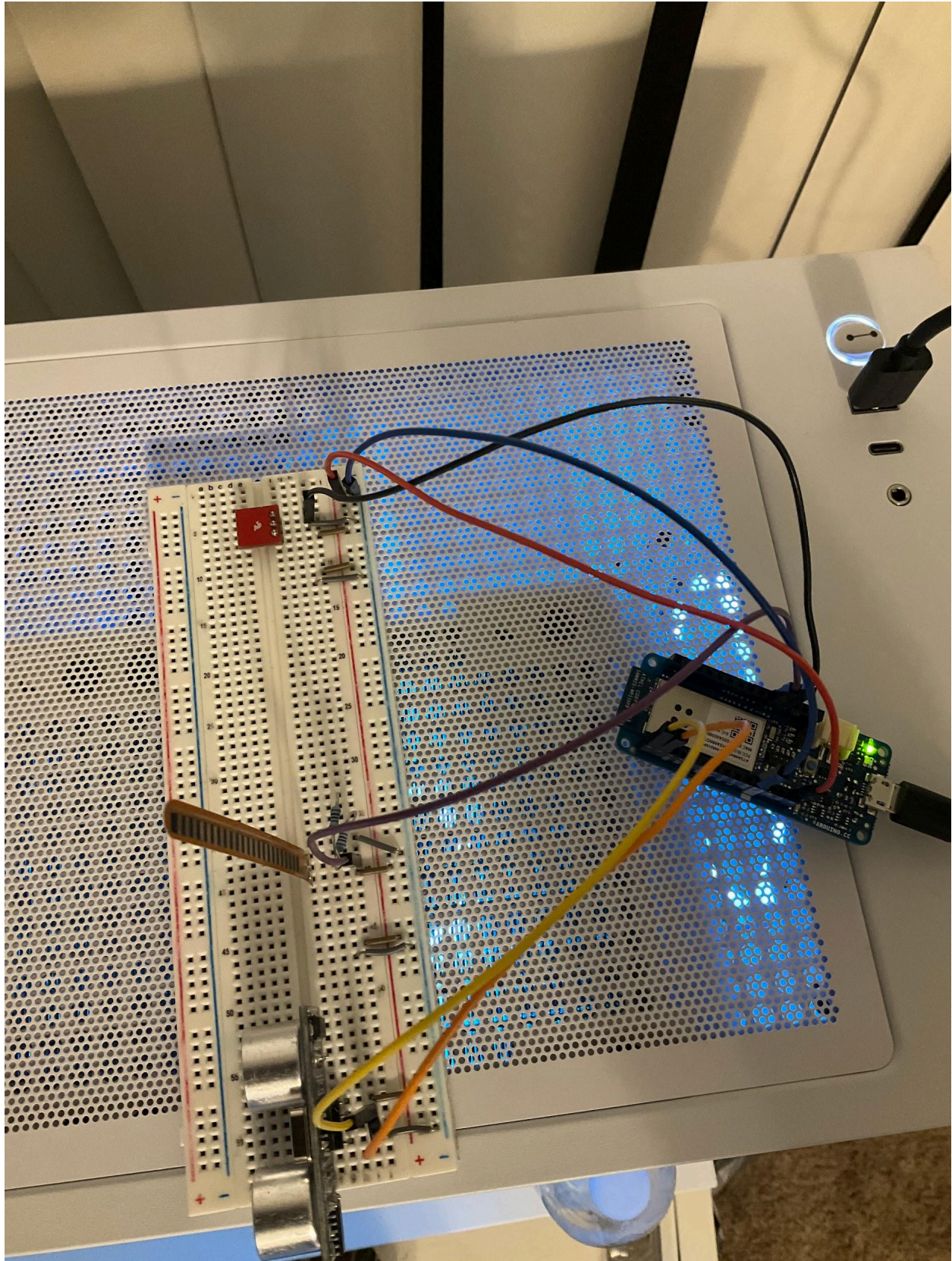
```
21:13:38.476 -> Received a message with topic '  
21:13:38.476 -> sensors/light', length 4 bytes:  
21:13:38.476 -> Light: 22  
21:13:38.476 ->  
21:13:38.476 ->  
21:13:38.476 ->  
21:13:46.427 -> Received a message with topic '  
21:13:46.427 -> sensors/distance', length 8 bytes:  
21:13:46.427 -> Distance: 114.31  
21:13:46.427 ->  
21:13:46.427 ->  
21:13:46.427 ->  
21:13:46.471 -> Received a message with topic '  
21:13:46.471 -> sensors/flex', length 4 bytes:  
21:13:46.471 -> Flex: 22  
21:13:46.471 ->  
21:13:46.471 ->  
21:13:46.471 ->  
21:13:46.471 -> Received a message with topic '  
21:13:46.471 -> sensors/light', length 4 bytes:  
21:13:46.471 -> Light: 24  
21:13:46.471 ->  
21:13:46.471 ->  
21:13:46.471 ->
```

c. Log of the channels activity (if broker permits)

IoT Systems – Final project



IoT Systems – Final project



4. Describe technical challenges you had and how you solved them

Some issues I had was getting the MQTT protocol to work along with the ThingSpeak. Sometimes my connection would connect or disconnect and outright not send data to the cloud. I included if statements to check the connection in both the loop and setup so debugging was easier. What I ended up doing is starting over from scratch. I started back to square one when my sensor readings were working. I also originally wanted to include more sensors before but stuck with 3 sensors because the others weren't working correctly. I started working little by little like focusing on the MQTT code and making it nice and simple to compile. Once I was able to get it to work I moved on to getting the Blynk code to work. This process didn't take too long as I was able to get it to work correctly. Lastly, the thingspeak took a couple tries to get it to work but ultimately what I had to do was remove each individual function that my sensors had because it was causing such a huge delay. However, I simplified my code and was able to get it functioning. If given more time I would also like to implement the buzzer as the action when all sensors reach a certain threshold. This would improve the IoT system and make it more efficient.

5. Video

https://youtube.com/shorts/noh5ysa34Bw?si=L2CGEWpU_N094LPH

6. Code listings for each node and cloud codes

Publisher Code:

```
//publisher code
#include "ThingSpeak.h"
#include <ArduinoMqttClient.h>
#include <WiFi101.h>
#define BLYNK_TEMPLATE_ID "TMPL2KIo5FwVI"
#define BLYNK_TEMPLATE_NAME "Kimberly"
#define BLYNK_AUTH_TOKEN "9hP9yGXO3NndpLtPTAjbZ_Ekuo-yn7BH"
#define BLYNK_PRINT SerialUSB
#include <BlynkSimpleWiFiShield101.h>
#include <SPI.h>
#define lightPin A2
#define flexPin A3
char auth[] = "9hP9yGXO3NndpLtPTAjbZ_Ekuo-yn7BH";

const int echoPin = 7;
const int trigPin = 6;
```

IoT Systems – Final project

```
long duration = 0.0;
float distance = 0.0;
int lightval = 0;
int data = 0;

unsigned long myChannelNumber = 2784201;
const char * myWriteAPIKey = "88FWY1MDQ6BNEGQL";

char ssid[] = "RojasFam";
char pass[] = "L0v3sh0t";
int status = WL_IDLE_STATUS;
WiFiClient wifiClient;
MqttClient mqttClient (wifiClient);
const char broker[] = "test.mosquitto.org";
int port = 1883;

const char topicDistance[] = "sensors/distance";
const char topicFlex[] = "sensors/flex";
const char topicLight[] = "sensors/light";

const long interval = 8000;
unsigned long prevMill = 0;
int count = 0;

void setup() {
    // put your setup code here, to run once:
    // Debug console
    Serial.begin(9600);
    while (!Serial) {
        ; // wait for serial port to connect. Needed for native USB port only
    }

    // attempt to connect to Wifi network:
    Serial.print("Attempting to connect to WPA SSID: ");
    Serial.println(ssid);
    while (WiFi.begin(ssid, pass) != WL_CONNECTED) {
        // failed, retry
        Serial.print(".");
        delay(5000);
    }
}
```

IoT Systems – Final project

```
Serial.println("You're connected to the network");
Serial.println();
//connection to the broker
Serial.print("Attempting to connect to the MQTT broker: ");
Serial.println(broker);
//connection to the broker failed
if (!mqttClient.connect(broker, port)) {
Serial.print("MQTT connection failed! Error code = ");
Serial.println(mqttClient.connectError());
while (1);
}
Serial.println("You're connected to the MQTT broker!");
Serial.println();
Blynk.begin(auth, ssid, pass);
ThingSpeak.begin(wifiClient);
pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);
pinMode(flexPin, INPUT);
pinMode(lightPin, INPUT);
delay(15000);

}

void loop() {
  Blynk.run();
  status = WiFi.begin(ssid, pass);
  mqttClient.poll();
  unsigned long currentMillis = millis();
  if (currentMillis - prevMill >= interval) {
    // save the last time a message was sent
    prevMill = currentMillis;
    //acquire distance
    digitalWrite(trigPin, LOW);
    delay(1);
    digitalWrite(trigPin, HIGH);
    delay(1);
    digitalWrite(trigPin, LOW);

    //distance calculation
    duration = pulseIn(echoPin, HIGH);
```

IoT Systems – Final project

```
distance = (duration * .034) /2;

//prints the distance

if(distance <= 30)
{
    Serial.print("unknown object in proximity.");
}
Serial.print("Sending message to topicDistance: ");
Serial.println(topicDistance);
Serial.println(distance);
mqttClient.beginMessage(topicDistance);
mqttClient.println(distance);
mqttClient.endMessage();
Blynk.virtualWrite(V1, distance);
ThingSpeak.setField(1, distance);

data = analogRead(flexPin);
if( data < 10 )
{
    Serial.println("object detected on floor");
}

Serial.print("Sending message to topicFlex: ");
Serial.println(topicFlex);
Serial.println(data);
mqttClient.beginMessage(topicFlex);
mqttClient.println(data);
mqttClient.endMessage();
Blynk.virtualWrite(V3, data);
ThingSpeak.setField(2, data);

lightval = analogRead(lightPin);
if(lightval > 20 )
{
    Serial.println("Suspicious activity");
    Serial.print("Light: ");
    Serial.println(lightval);
}
```

IoT Systems – Final project

```
Serial.print("Sending message to topicLight: ");
Serial.println(topicLight);
Serial.println(lightval);
mqttClient.beginMessage(topicLight);
mqttClient.println(lightval);
mqttClient.endMessage();
  Blynk.virtualWrite(V2, lightval);
  ThingSpeak.setField(3, lightval);
  ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
}
}
```

Client Code:

```
//client code
#include <ArduinoMqttClient.h>
#include <WiFi101.h>
//WiFi that you want to be connected
char ssid[] = "RojasFam"; // your network SSID
char pass[] = "L0v3sh0t"; // your network password
WiFiClient wifiClient;
MqttClient mqttClient(wifiClient);
const char broker[] = "test.mosquitto.org"; //your broker here is
mosquitto
int port = 1883; //channel of the broker
//topics, you can change the name
const char topicDistance[] = "sensors/distance";
const char topicFlex[] = "sensors/flex";
const char topicLight[] = "sensors/light";
void setup()
{
  //Initialize serial and wait for port to open:
  Serial.begin(9600);
  while (!Serial) {
```

IoT Systems – Final project

```
;
}
// attempt to connect to Wifi network:
Serial.print("Attempting to connect to SSID: ");
Serial.println(ssid);
while (WiFi.begin(ssid, pass) != WL_CONNECTED) {
  // failed, retry
  Serial.print(".");
  delay(5000);
}
Serial.println("You're connected to the network");
Serial.println();
Serial.print("Attempting to connect to the MQTT broker: ");
Serial.println(broker);
if (!mqttClient.connect(broker, port)) {
  Serial.print("MQTT connection failed! Error code = ");
  Serial.println(mqttClient.connectError());
  while (1);
}
Serial.println("You're connected to the MQTT broker!");
Serial.println();

mqttClient.subscribe(topicDistance);
mqttClient.subscribe(topicFlex);
mqttClient.subscribe(topicLight);

Serial.print("Subscribing to topics: ");
Serial.println(topicDistance);
Serial.println(topicFlex);
Serial.println(topicLight);
Serial.println();
mqttClient.onMessage(onMqttMessage);
Serial.println();
}

void loop() {
  mqttClient.poll();
}

//print the MQTT message
void onMqttMessage(int messageSize) {
  String topic = mqttClient.messageTopic();
```


IoT Systems – Final project

```
String message = "";

while(mqttClient.available())
{
    message += (char)mqttClient.read();
}

Serial.println("Received a message with topic ");
Serial.print(topic);
Serial.print(", length ");
Serial.print(messageSize);
Serial.println(" bytes:");

if(topic == "sensors/distance")
{
    Serial.print("Distance: ");
    Serial.println(message);
}
else if( topic == "sensors/flex")
{
    Serial.print("Flex: ");
    Serial.println(message);
}
else{
    Serial.print("Light: ");
    Serial.println(message);
}

while (mqttClient.available()) {
    Serial.print((char)mqttClient.read());
}

Serial.println();
Serial.println();
}
```

